✅ ALTIS – Phase 1:

Normalization to 3NF & Documentation

🔹 **Normalization Steps to 3NF**

Below is a walkthrough showing how we applied **1NF → 2NF → 3NF** normalization to your schema.

🎯 **First Normal Form (1NF)**

- Each table has atomic values (no repeating or composite groups).
- Example: In STUDENTS, only one contact per record. In COURSES, one course per row.

✅ All relations meet **1NF**.

🎯 **Second Normal Form (2NF)**

- All non-key attributes must depend on the **whole** primary key (especially for composite keys).
- This applies only to tables where the **primary key is composite**.

In your schema:

- All tables have single-attribute primary keys (e.g., StudentID, ClassroomID), so **no partial dependencies exist**.

✅ All relations are already in **2NF**.

🎯 **Third Normal Form (3NF)**

- No transitive dependencies (non-key → non-key via another non-key).

- Every non-prime attribute depends **only** on the primary key.

✅ Verified for each table:

Table	3NF Explanation
ADMINS	All fields depend only on AdminID
FACULTY	All fields depend only on FacultyID
STUDENTS	All attributes depend only on StudentID
COURSES	CourseName depends only on CourseID
SECTIONS	SectionName depends only on SectionID
COURSE_ASSIGNMENTS	Fully normalized; all attributes depend on AssignmentID
ENROLLMENTS	Only contains FKs; all depend on EnrollmentID
CLASSROOMS	All fields relate to ClassroomID and have no transitive dependencies
ASSIGNMENTS	Fully dependent on AssignmentID
SUBMISSIONS	All depend on SubmissionID
ATTENDANCE	Status, Date depend only on AttendanceID
MARKS	Score depends only on MarkID
RESULTS	GPA, Remarks depend only on ResultID

✅ **All tables meet 3NF**

📄 **Schema Design Documentation**

🔸 **Schema Design Decisions**

1. **Separation of Roles**: ADMINS, FACULTY, and STUDENTS are split to allow role-specific attributes and access control.
2. **No Composite Keys**: All tables use **single-column primary keys** for clarity and easier indexing.
3. **Bridge Tables**:
   - ENROLLMENTS: Handles many-to-many between STUDENTS and CLASSROOMS
   - COURSE_ASSIGNMENTS: Connects FACULTY, COURSES, and SECTIONS
4. **Normalized Content Delivery**:
   - CLASSROOMS encapsulates teaching logic.
   - ASSIGNMENTS and SUBMISSIONS ensure decoupling of evaluation from classroom itself.
5. **Results/Marks Separation**: RESULTS summarize academic status; MARKS give per-assignment performance.

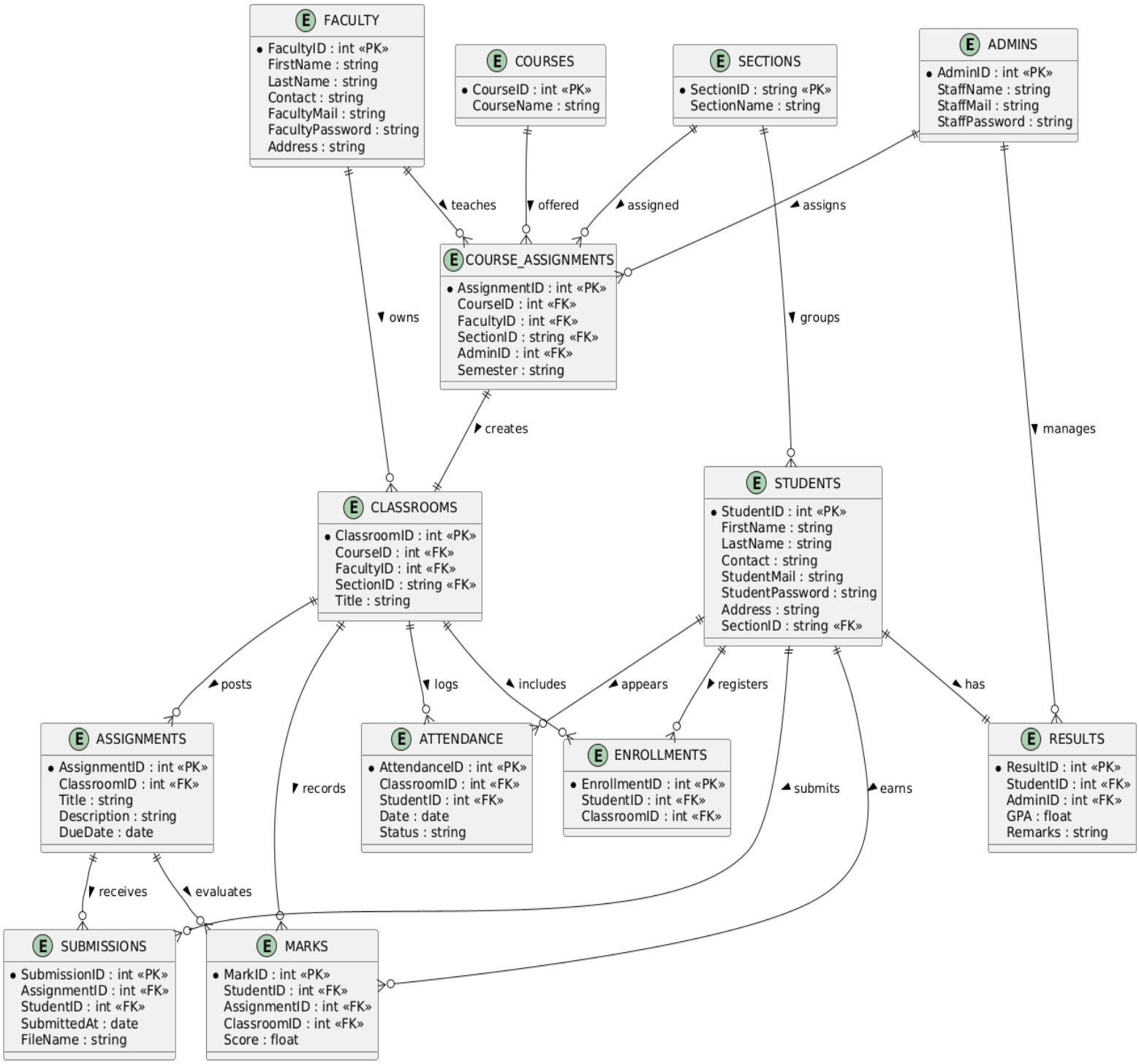
🔸 **Entity-Attribute Relationships**

Entity	Key Attributes & Purpose
ADMINS	AdminID is the authority who assigns courses and manages results
FACULTY	Faculty profile and login data

STUDENTS	Student profile, contact, and section assignment
COURSES	Course catalog with unique CourseID
SECTIONS	Groups of students based on time/class division
COURSE_ASSIGNMENTS	Assigns Faculty + Course + Section, done by admin
CLASSROOMS	A teaching space for the above combo
ASSIGNMENTS	Course work posted in classrooms
SUBMISSIONS	Records students' submitted assignments
ENROLLMENTS	Tracks which student joined which classroom
ATTENDANCE	Tracks student presence per class session
MARKS	Numeric scores per assignment per class
RESULTS	Semester-end summary; managed by admin

- **Design Assumptions**

1. Each student is enrolled in **only one section**.
2. Each classroom is created **per course-section assignment**, **owned by one faculty**.
3. A student may be enrolled in **multiple classrooms** (via ENROLLMENTS) but each enrollment links to one CLASSROOMID.
4. One COURSE_ASSIGNMENT creates one CLASSROOM, which houses ASSIGNMENTS.

5. SUBMISSIONS map ASSIGNMENTID and STUDENTID (many-to-many via rows).
6. RESULTS are generated once per student per semester by the ADMIN.

Enhanced Entity Relationship Diagram (EERD)